

harini0389-del / study-of-basic-gates

<> Code

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

0 stars

2k forks

0 watching

Branches

Activity

Tags

Public repository · Forked from [naavaneetha/study-of-basic-gates](#)

1 Branch

0 Tags

Go to file

Go to file

Add file

Code

This branch is 1 commit ahead of naavaneetha/study-of-basic-gates:main .

Contribute

Sync fork

harini0389-del

Update README.md

ba7a477 · 3 days ago

db	project created	last year
hc_output	project created	last year
incremental_db	project created	last year
output_files	project created	last year
simulation	project created	last year
GATES_TT.qpf	project created	last year
GATES_TT.qsf	project created	last year
GATES_TT.v	project created	last year
GATES_TT.v.bak	project created	last year
README.md	Update README.md	3 days ago
Waveform1a.vwf	project created	last year
c5_pin_model_dump.txt	project created	last year
cio_dump_disallowed_lists.e...	project created	last year

README

study-of-basic-gates

AIM:

To study and verify the truth table of logic gates in Quartus II using Verilog programming.

Equipments Required:

Software – Quartus prime

Theory

Introduction Logic gates are the basic building blocks of any digital system. Logic gates are electronic circuits having one or more than one input and only one output. The relationship between the input and the output is based on a certain logic. Based on this, logic gates are named as

AND gate OR gate NOT gate NAND gate NOR gate Ex-OR gate Ex-NOR gate

AND gate

The AND gate is an electronic circuit that gives a high output (1) only if all its inputs are high. A dot (.) is used to show the AND operation i.e. $A.B$ or can be written as AB $Y = A.B$

OR gate

The OR gate is an electronic circuit that gives a high output (1) if one or more of its inputs are high. A plus (+) is used to show the OR operation. $Y = A+B$

NOT gate

The NOT gate is an electronic circuit that produces an inverted version of the input at its output. It is also known as an inverter. If the input variable is A, the inverted output is known as NOT A. This is also shown as A' or A with a bar over the top, as shown at the outputs. $Y = A'$

NAND gate

This is a NOT-AND gate which is equal to an AND gate followed by a NOT gate. The outputs of all NAND gates are high if any of the inputs are low. The symbol is an AND gate with a small circle on the output. The small circle represents inversion. $Y = (AB)'$

NOR gate

This is a NOT-OR gate which is equal to an OR gate followed by a NOT gate. The outputs of all NOR gates are low if any of the inputs are high. The symbol is an OR gate with a small circle on the output. The small circle represents inversion. $Y = (A+B)'$

Ex-OR gate

The 'Exclusive-OR' gate is a circuit which will give a high output if either, but not both of its two inputs are high. An encircled plus sign ($\oplus$) is used to show the Ex-OR operation. $Y = A \oplus B$

Ex-NOR gate

The 'Exclusive-NOR' gate circuit does the opposite to the EX-OR gate. It will give a low output if either, but not both of its two inputs are high. The symbol is an EX-OR gate with a small circle on the output. The small circle represents inversion. $Y = A \oplus B$

Procedure

1. Type the program in Quartus software.
2. Compile and run the program.
3. Generate the RTL schematic and save the logic diagram.
4. Create nodes for inputs and outputs to generate the timing diagram.

5. For different input combinations generate the timing diagram.

PROGRAM module logicgates1(a, b, c); input a; input b; output [6:0] c;

```
assign c[0]= a & b;  
assign c[1]= a | b;  
assign c[2]= ~(a & b);  
assign c[3]= ~(a | b);  
assign c[4]= a ^ b;  
assign c[5]= ~(a ^ b);  
assign c[6]= ~ a;
```

endmodule

Program for logic gates and verify its truth table in quartus using Verilog programming

Developed by:Harini M RegisterNumber:25001011



Logic symbol & Truth table

LOGIC DIAGRAM:

AND GATE:

LOGIC DIAGRAM:



TRUTH TABLE:

A	B	Y=AB
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE:

LOGIC DIAGRAM

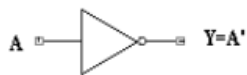


TRUTH TABLE:

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE:

LOGIC DIAGRAM:



TRUTH TABLE:

A	Y=A'
0	1
1	0

NAND GATE:

LOGIC DIAGRAM



TRUTH TABLE

A	B	Y=(AB)'
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE:

LOGIC DIAGRAM:

TRUTH TABLE:

A	B	Y=(A+B)'
0	0	1
0	1	0
1	0	0

XOR GATE:

LOGIC DIAGRAM

TRUTH TABLE



1	1	0
---	---	---



A	B	Y=A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

XNOR GATE:

LOGIC DIAGRAM:



TRUTH TABLE:

A	B	Y=A⊙B
0	0	1
0	1	0
1	0	0
1	1	1

RTL realization Output:

c[2]~not

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

● VHDL 54.4% ● HTML 14.9% ● Stata 14.7% ● Verilog 13.3% ● Standard ML 2.7%

Suggested workflows

Based on your tech stack



SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows

Configure



Jekyll using Docker image

Package a Jekyll site using the jekyll/builder Docker image.

Configure

[More workflows](#)

[Dismiss suggestions](#)